1. Introduction to API

An **Application Programming Interface (API)** is a formal set of rules, protocols, and definitions that allows two or more software applications to communicate and exchange data seamlessly without needing to know each other's internal source code implementation.

2. Why Do We Need APIs?

APIs act as standardized connectors in modern software architecture, offering crucial benefits:

- **System Communication:** Enables independent applications to transfer structured datasets safely.
- **Data Sharing & Abstraction:** Exposes necessary endpoints while safeguarding underlying database logic.
- **Third-Party Integration:** Incorporates robust services like payment processors (Razorpay, Stripe) and maps (Google Maps).
- **Automation & Reusability:** Prevents reinventing core functionalities by consuming existing microservices.

3. Real-World Examples

Application Area	Real-World Usage Example
Weather Forecasting	Retrieves live atmospheric data from OpenWeatherMap API.
E-Commerce	Processes customer transactions securely via Razorpay payment gateway API.
AI & Automation	Sends prompt payloads to LLM endpoints and receives structured responses for Muskan.

4. Basic API Request-Response Cycle



> **Key Elements of API Architecture:**
> A standard API call comprises an **Endpoint URL**, **HTTP Method**, **Headers** (for authentication & format specification), and optional **Request Body** payload (JSON/XML).

2. Detailed API Architecture & Communication

APIs utilize the client-server architecture model operating over standard Internet protocols (HTTP/HTTPS) to perform stateless computational transactions.

Structure of an HTTP Request

```
GET /api/v1/users/muskan HTTP/1.1
Host: api.example.com
Authorization: Bearer bearer_token_xyz123
Content-Type: application/json
Accept: application/json

{
  "user": "Muskan Shaikh"
}
```

Structure of an HTTP Response

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 112

{
  "status": "success",
  "data": {
    "id": 101,
    "name": "Muskan Shaikh",
    "email": "muskan@example.com"
  }
}
```

Real Request Example (cURL Command)

```
curl -X GET "https://api.example.com/users/muskan" \
  -H "Accept: application/json" \
  -H "Authorization: Bearer token123"
```

> **Statelessness Principle:**
> HTTP APIs are inherently **stateless**. Every individual request sent by the client must carry all necessary context, including tokens and query parameters, as the server retains no session memory between calls.

3. HTTP Methods & Status Codes

Primary HTTP Verbs

Method	Purpose	Idempotent?	Example Endpoint
GET	Retrieve existing resources from server.	Yes	`GET /api/students`
POST	Create a new resource on server.	No	`POST /api/students`
PUT	Completely replace/update a resource.	Yes	`PUT /api/students/101`
DELETE	Remove a specified resource.	Yes	`DELETE /api/students/101`

HTTP Status Code Categories

Code Range	Category	Common Examples & Description
1xx	Informational	`100 Continue` - Request accepted, proceed.
2xx	Success	`200 OK`, `201 Created` - Action completed successfully.
3xx	Redirection	`301 Moved Permanently` - Endpoint URL shifted.
4xx	Client Error	`400 Bad Request`, `401 Unauthorized`, `404 Not Found`.
5xx	Server Error	`500 Internal Server Error`, `503 Service Unavailable`.

Python Code: Handling Status Codes

```
import requests

response = requests.get("https://api.example.com/users/muskan")
if response.status_code == 200:
    print("User Data:", response.json())
elif response.status_code == 404:
    print("Error: User Muskan Shaikh not found.")
else:
    print("HTTP Error Status:", response.status_code)
```

Application Programming Interface (API) ✨ Notes by Muskan Shaikh

4. Types of APIs (Architecture & Protocol)

Architectural Patterns

Architecture	Data Format	Key Characteristic	Typical Use Case
REST	JSON / XML	Stateless, resource-based URLs.	Web applications & mobile clients.
SOAP	XML only	Strict protocol standards (WSDL).	Enterprise banking & financial systems.
GraphQL	JSON	Single endpoint, query-based fetching.	Modern frontend frameworks (React).
gRPC	Protobuf	High performance over HTTP/2.	Microservice-to-microservice calls.

Advanced Concepts: Webhooks vs Polling, Idempotency & CORS

> **1. Real-Time Communication: Polling vs Webhooks**
> **Polling:** Client repeatedly pings server at fixed intervals asking "Is new data available for Muskan?". High overhead.
> **Webhooks:** Reverse API pattern where the server sends an automated HTTP POST payload to the client's listener URL as soon as an event occurs.

2. Idempotency in API Operations
An API method is **idempotent** if executing it multiple times produces the exact same outcome as executing it once. `GET`, `PUT`, and `DELETE` are idempotent; `POST` is not.

3. Cross-Origin Resource Sharing (CORS)
Browser security mechanism enforcing HTTP headers (e.g., `Access-Control-Allow-Origin`) to grant web applications running at one domain permission to access resources from a server at another domain.

5. API Security & OAuth 2.0

Common Security Threats

- **Unauthorized Access:** Exposing endpoints without credential validation.
- **Data Leakage:** Returning sensitive fields (e.g., password hashes) in API payloads.
- **Injection Attacks:** SQL or command injection via query string parameters.

Authentication Mechanisms

Mechanism	Implementation Method	Use Case
API Key	Passed via header: `x-api-key: secret_key_123`	Simple public API access.
JWT (Bearer)	Header: `Authorization: Bearer <token>`	Stateless user sessions.
OAuth 2.0	Delegated token flow via authorization server.	Google / GitHub Login integration.

OAuth 2.0 Sequence Flow



Security Best Practices:
1. Always mandate HTTPS. 2. Implement Rate Limiting headers (`X-RateLimit-Limit`). 3. Validate and sanitize inputs to prevent injection.

6. Consuming APIs in Python & Java

Python Code (Using Requests)

```
import requests

url = "https://jsonplaceholder.typicode.com/posts"
payload = {"title": "API Study", "body": "Notes for Muskan Shaikh", "userId": 101}
headers = {"Content-Type": "application/json"}

try:
    response = requests.post(url, json=payload, headers=headers, timeout=5)
    response.raise_for_status()
    print("Python POST Status:", response.status_code)
    print("Response JSON:", response.json())
except requests.exceptions.RequestException as e:
    print("Error:", e)
```

Java Code (Using Java 11+ HttpClient)

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class ApiConsumer {
    public static void main(String[] args) {
        try {
            HttpClient client = HttpClient.newHttpClient();
            HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create("https://jsonplaceholder.typicode.com/users/1"))
                .header("Accept", "application/json")
                .GET()
                .build();

            HttpResponse<String> response = client.send(request,
HttpResponse.BodyHandlers.ofString());
            System.out.println("Java GET Status: " + response.statusCode());
            System.out.println("Response Body: " + response.body());
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

7. Building REST APIs (Flask & Java Spring Boot)

Python Implementation (Flask Framework)

```
from flask import Flask, jsonify, request

app = Flask(__name__)
students = [{"id": 1, "name": "Muskan Shaikh"}]

@app.route('/api/students', methods=['GET'])
def get_students():
    return jsonify(students)

@app.route('/api/students', methods=['POST'])
def add_student():
    data = request.get_json()
    new_student = {"id": len(students) + 1, "name": data['name']}
    students.append(new_student)
    return jsonify(new_student), 201

if __name__ == '__main__':
    app.run(debug=True)
```

Java Implementation (Spring Boot Controller)

```
@RestController
@RequestMapping("/api/students")
public class StudentController {

    private List<Student> studentList = new ArrayList<>(
        List.of(new Student(1, "Muskan Shaikh"))
    );

    @GetMapping
    public List<Student> getAllStudents() {
        return studentList;
    }

    @PostMapping
    public ResponseEntity<Student> createStudent(@RequestBody Student student) {
        studentList.add(student);
        return new ResponseEntity<>(student, HttpStatus.CREATED);
    }
}
```

8. API Testing & Debugging with Postman

Postman is an industry-standard API client platform used for development testing, automated test suite execution, and request debugging.

Testing Steps in Postman

1. Select the HTTP request method (`GET`, `POST`, etc.).
2. Enter the target endpoint URL (e.g., `https://api.example.com/v1/students`).
3. Set Request Headers (`Authorization`, `Content-Type: application/json`).
4. Provide the raw JSON body payload for write requests.
5. Click **Send** and inspect status code, response body, and headers.

Common Error Handling Codes

Status Code	Meaning	Root Cause & Solution
`400 Bad Request`	Malformed Syntax	Check JSON body payload format for missing quotes/commas.
`401 Unauthorized`	Authentication Failed	Missing or expired Bearer token in request header.
`404 Not Found`	Missing Resource	Endpoint URL path is incorrect.
`500 Internal Error`	Server Exception	Unhandled application crash on backend service.

Application Programming Interface (API) ✨ Notes by Muskan Shaikh

9. API Classification Matrix

Access-Level Classification

Category	Accessibility	Example Endpoint / Service
Public API	Open access for developer community.	OpenWeatherMap API, Twitter API
Private API	Strictly internal organization systems.	Internal HR portal microservices
Partner API	Shared with authorized external vendors.	Razorpay B2B Merchant Payment API
Composite API	Aggregates multiple underlying API endpoints.	Batch user profile + orders response call

System-Level Classification

Type	Description	Real-World Usage
Web API	Accessed over HTTP protocol.	RESTful microservices
Library API	Provided by language frameworks.	Python `requests`, Java `java.net.http`
Hardware API	Interacts with mobile device hardware.	Android Camera2 API, Location GPS API

10. Comparative Analysis: REST vs SOAP vs GraphQL

Feature	REST API	SOAP API	GraphQL API
Protocol	HTTP / HTTPS	HTTP, SMTP, TCP	HTTP
Data Format	JSON (mainly), XML	XML strictly	JSON strictly
Performance	Lightweight, fast	Slower, heavy payload	Highly efficient
Over-fetching	Possible	Possible	Eliminated completely
Best For	Web/Mobile apps	Enterprise Banking	Complex modern UI

Syntax Comparison

REST Request (JSON Output)

```
GET https://api.example.com/users/muskan
```

GraphQL Query (Selective Fetching)

```
query {
  user(name: "Muskan Shaikh") {
    email
  }
}
```

Application Programming Interface (API) ✨ Notes by Muskan Shaikh

11. Deep Dive: RESTful API Principles

Representational State Transfer (REST) is an architectural style designed by Roy Fielding around 6 core constraints:

- **Uniform Interface:** Uniform URI endpoint design for data resource access.
- **Statelessness:** No client state stored on server between requests.
- **Cacheability:** Responses must define themselves as cacheable or non-cacheable.
- **Client-Server Separation:** UI concern separated from data storage logic.
- **Layered System:** Client cannot tell whether connected directly to end server or proxy.

REST Resource Mapping Standard

```
GET    /api/v1/students          -> Retrieve all student records
GET    /api/v1/students/101      -> Retrieve student ID 101 (Muskan Shaikh)
POST   /api/v1/students          -> Create a new student entry
PUT    /api/v1/students/101      -> Overwrite student record 101
DELETE /api/v1/students/101      -> Delete student entry 101
```

12. Deep Dive: SOAP API Protocol

Simple Object Access Protocol (SOAP) relies strictly on XML messaging enveloped inside formal structures defined by Web Services Description Language (WSDL) contracts.

Structure of a SOAP Message

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
                  xmlns:mus="http://example.com/muskan">
   <soapenv:Header/>
   <soapenv:Body>
      <mus:GetUserRequest>
         <mus:userId>101</mus:userId>
      </mus:GetUserRequest>
   </soapenv:Body>
</soapenv:Envelope>
```

SOAP XML Response

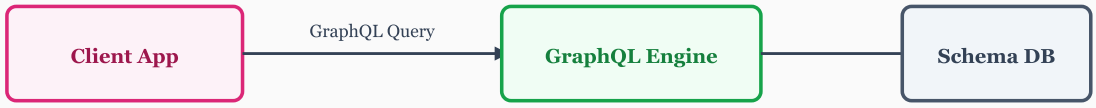
```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/">
   <soapenv:Body>
      <GetUserResponse>
         <id>101</id>
         <name>Muskan Shaikh</name>
      </GetUserResponse>
   </soapenv:Body>
</soapenv:Envelope>
```

Application Programming Interface (API) ✨ Notes by Muskan Shaikh

13. Deep Dive: GraphQL Architecture

GraphQL exposes a declarative query language operating through a single `/graphql` endpoint, allowing clients to request exact fields.

GraphQL Server Architecture Diagram



GraphQL Request and Response Payload

```
// Request Query
query {
  student(id: "101") {
    name
    email
  }
}

// JSON Response Received
{
  "data": {
    "student": {
      "name": "Muskan Shaikh",
      "email": "muskan@example.com"
    }
  }
}
```

14. Real-World API Code Examples

1. Public Geocoding API Call (Google Maps)

```
GET https://maps.googleapis.com/maps/api/geocode/json?address=India&key=YOUR_API_KEY
```

2. Partner Payment API Request (Razorpay Order)

```
POST https://api.razorpay.com/v1/orders
Headers:
  Content-Type: application/json
  Authorization: Basic key_secret_token

Body:
{
  "amount": 50000,
  "currency": "INR",
  "receipt": "order_muskan_101"
}
```

3. Composite API Aggregation Response

```
GET https://api.example.com/v1/user-summary?user=muskan

Response:
{
  "profile": { "name": "Muskan Shaikh" },
  "orders": [ { "order_id": 991, "amount": 1499 } ],
  "notifications": { "unread": 2 }
}
```

Application Programming Interface (API) ✨ Notes by Muskan Shaikh

15. Step-by-Step API Integration Workflow

Workflow Steps

1. **Select Target API:** Choose appropriate provider (e.g., OpenWeatherMap).
2. **Read Official Documentation:** Review endpoint paths, mandatory parameters, rate limits, and error handling.
3. **Acquire Credentials:** Obtain secret API Key / Client Secret token.
4. **Perform Test Calls:** Validate URL queries using Postman client.
5. **Implement Code Logic:** Build application calls using Python / Java / JavaScript.
6. **Handle Errors Gracefully:** Wrap API invocations in try-catch blocks with retry policies.

Sample Live API Query Parameter Structure

```
Endpoint: https://api.openweathermap.org/data/2.5/weather
Parameters:
  - q: City Name
  - appid: YOUR_API_KEY
  - units: metric
```

> **Important Takeaway:**
> Never hardcode API Keys in client-side code repository. Always store secret credentials inside secure server-side environment variables (e.g., `.env` file).

16. Complete End-to-End Weather Project

Python Implementation

```
import requests

API_KEY = "YOUR_API_KEY"
city = "Delhi"
url = f"https://api.openweathermap.org/data/2.5/weather?q={city}&appid={API_KEY}&units=metric"

try:
    response = requests.get(url, timeout=5)
    data = response.json()
    print(f"Weather Report for: {data['name']}")
    print(f"Temperature: {data['main']['temp']} °C")
    print(f"Humidity: {data['main']['humidity']} %")
    print(f"Condition: {data['weather'][0]['description']}")
except Exception as e:
    print("Execution Error:", e)
```

JavaScript (Browser Fetch API Implementation)

```
const apiKey = "YOUR_API_KEY";
const city = "Delhi";

fetch(`https://api.openweathermap.org/data/2.5/weather?
q=${city}&appid=${apiKey}&units=metric`)
  .then(response => response.json())
  .then(data => {
    console.log("City:", data.name);
    console.log("Temperature:", data.main.temp, "°C");
    console.log("Condition:", data.weather[0].description);
  })
  .catch(err => console.error("Error fetching weather:", err));
```

Key Revision Summary

- APIs provide standardized communication between independent applications.
- REST relies on HTTP verbs (GET, POST, PUT, DELETE) and JSON payload format.
- Security requires HTTPS, authentication tokens (JWT/OAuth), input validation, and rate limiting.